HARI ANITZEKO PROGRAMAZIOA





MULTI PROCESO

Prozesu anizkoitza hainbat prozesu aldi berean gauzatzean datza, elkarrekin zerikusia duten, edo ez, ataza bat, edo batzuk, egiteko. Kasu honetan, prozesu horietako bakoitza aplikazio independente bat da. Kasurik ezagunena sistema eragileari buruzkoa da (windows, linux, macOs). Multiataza dela esaten dugu, hainbat ataza edo prozesu (edo programa) aldi berean exekutatzeko gai dena.



MULTI HILO

Hari anitzekoaz ari gara aplikazio beraren barruan elkarrekin zerikusia duten, edo ez, ataza bat baino gehiago exekutatzen direnean. Kasu honetan, ez dira prozesu desberdinak, zeregin horiek sistema eragilearen prozesu beraren barruan gauzatzen dira. Zeregin horietako bakoitzari hari edo thread esaten zaio (testuinguru batzuetan, prozesu arinak ere esaten zaie).

Bi kasuetan, programazio konkurrentea esaten zaionari buruz ari gara. Kontuan izan behar da exekuzioa ez dela benetan aldiberekoa; izan ere, Sistema Eragileak egiten du horren itxura, baina planifikazio-algoritmoak esaten zaionari jarraituz exekutatzen ditu.

PLANGINTZA-ALGORITMOAK

Ataza anitzeko inguruneetan, plangintza-algoritmo batek adierazten du prozesatze-denbora nola banatu behar den une jakin batean gauzatu behar diren ataza guztien artean. Hainbat plangintza-algoritmo daude, bakoitza bere abantaila eta desabantailekin, baina denak saiatzen dira honako puntu hauek betetzen:

- Inpartziala eta eraginkorra izan behar du.
- Erabiltzaileari erantzuteko denbora minimizatu behar du, batez ere prozesu edo ataza elkarreragileenetan.
- Ahalik eta prozesu gehien gauzatu behar ditu.
- Sistemaren baliabideen erabileran oreka mantendu behar du.

FCFS: FIRST COME FIRST SERVED

Prozesadorera iristen den lehen prozesua lehenago eta osorik exekutatzen da. Exekuzioa amaitzen ez den arte, ezingo da beste prozesurik gauzatu.

RR: ROUND ROBIN

Txandakako algoritmo ere esaten zaio. Kasu horretan, zeregin guztiak prozesatzeko denbora-kopuru labur bat (quantum) izendatzen da. Prozesuan denbora gehiago behar dutenek beren txanda izan arte itxaron beharko dute exekutatzen jarraitzeko.

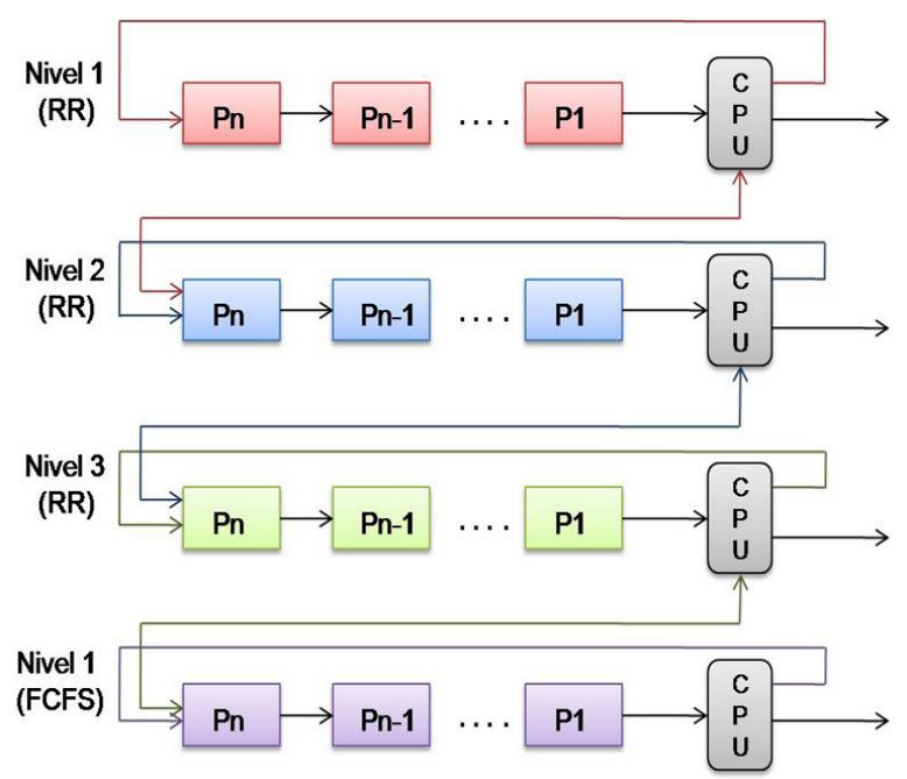
SPF: SHORTEST PROCESS FIRST

Algoritmo honetan, exekutatzeko prest dauden prozesu guztietatik, lehenengo laburrenak egingo du

SRT: SHORTEST REMAINING TIME

Gauzatzeko prest dauden prozesu guztien artean, amaitzeko denbora gutxien geratzen zaiona exekutatuko da.

BERRELIKADURA DUTEN HAINBAT ILARA



Aurreko algoritmo guztiak baino konplexuagoa da, eta, beraz, errealistagoa. Prozesu bat exekutatzen hastean zenbat denbora behar duen ez dakigun inguruneetan erabiltzen da. Kasu horretan, sistemak hainbat ilara ditu, eta horiek, aldi berean, politika desberdinak izan ditzakete. Prozesuak ilara batetik bestera igarotzen dira, exekutatzen amaitu arte. Kasu batzuetan, algoritmoa egokitu daiteke ilara-kopurua, politika eta abar aldatuz.

PROGRAMAZIO KONKURRENTEA, PARALELOA ETA BANATUA

Programazio konkurrentea

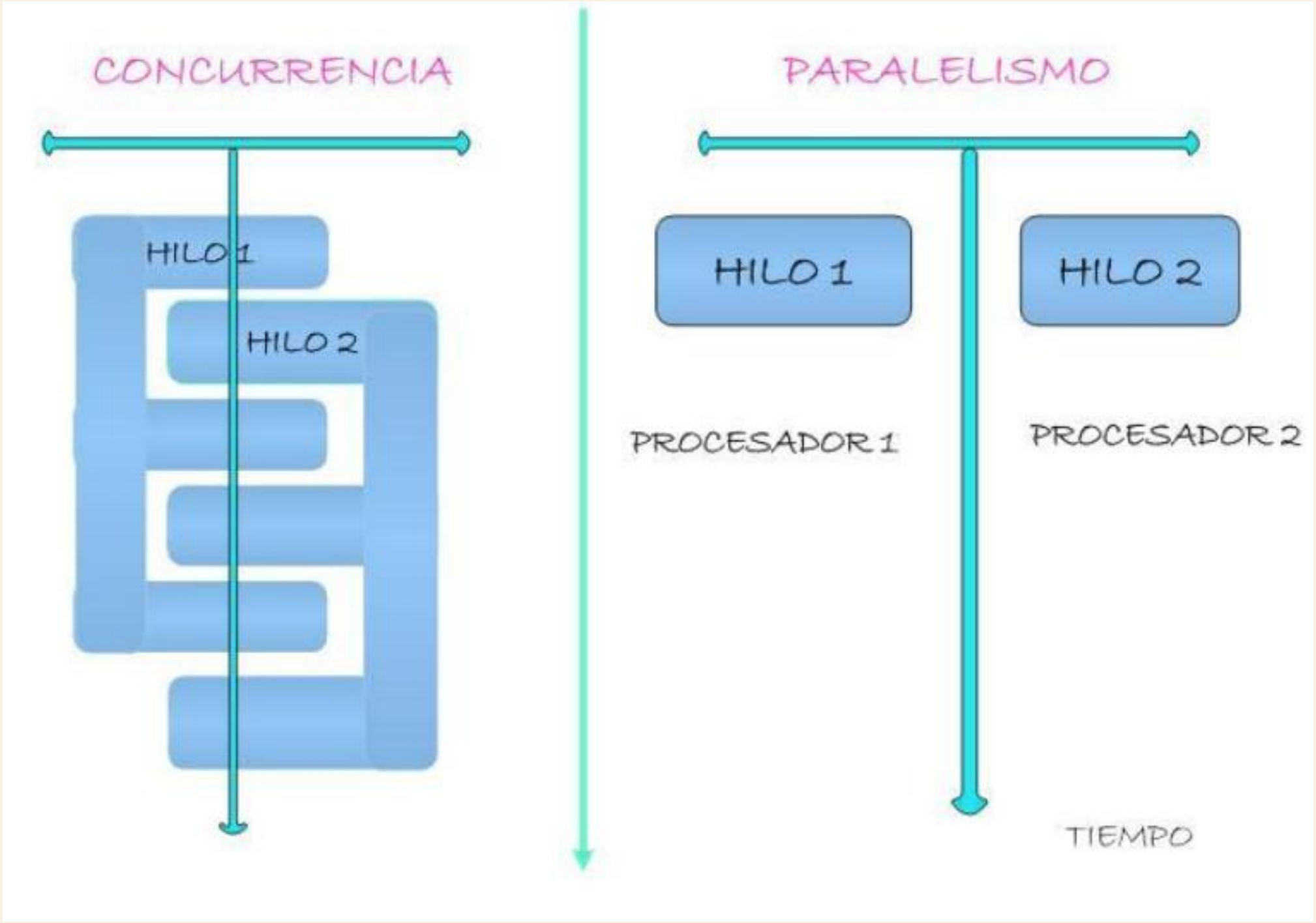
Hariak edo thread-ak erabiliz hainbat zeregin aldi berean egiteko gai diren aplikazioen programazioa da. Kasu honetan, zeregin guztiak prozesadorea erabiltzeagatik lehiatzen dira (ohikoena bat bakarrik edukitzea da), eta une jakin batean horietako bakarra dago exekutatzen. Gainera, kontuan izan behar da hainbat harik informazioa parteka dezaketela elkarren artean, eta horrek asko zailtzen duela programazioa eta koordinazioa.

Programazio paraleloa

Atazak aldi berean exekutatzen dituzten aplikazioen programazioa da, **prozesadorea lortzeko lehiatzen ez direnak, ataza bakoitza beste batean exekutatzen baita**. Normalean, emaitza komunak bilatzen dituzte, arazoa aldi berean exekutatzen diren hainbat zereginetan zatituz.

Programazio banatua

Gauzatu beharreko zereginak hainbat ekiporen artean banatzen dituzten aplikazioen programazioa da (sarean konektatuta daude, eta nodo deituko diegu). Elkarrekin, talde horiek Sistema Banatua osatzen dute, helburu komun batekin lan egingo duten ekipo-sareak osatzeko.

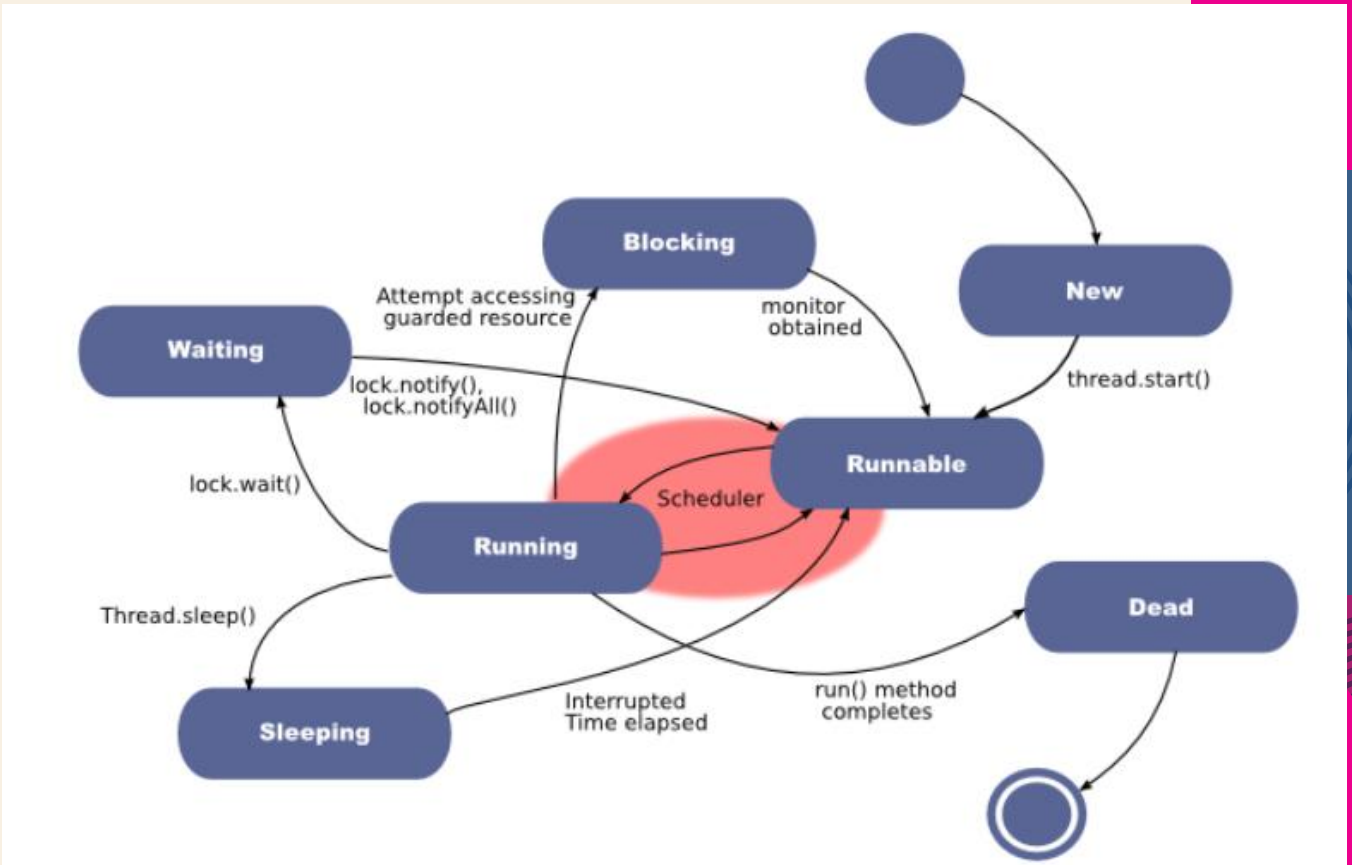

CONCURRENCIA
HILO 1
HILO 2
PARALELISMO
HILO 1
HILO 2
PROCESADOR 1
PROCESADOR 2
TIEMPO

ZER DIRA HARIAK?

Hari edo thread bat da aplikazio batek aldi berean egin dezakeen ataza bakoitza. Besterik adierazi ezean, aplikazio orok gauzatze-hari bakarra du, hari nagusi esaten zaiona. Aplikazio horrek ez badu beste haririk zabaltzen, hari nagusi horretan zeregin bat aldi berean egiteko gai izango da.

Horrela, aplikazio horretan gauzatu nahi den ataza gehigarri bakoitzerako, hari edo thread berri bat hasi beharko da. Horretarako, programazio-lengoaia guztiek, hala nola Javak, API bat dute sortzeko eta haiekin lan egiteko.

Nolanahi ere, oso garrantzitsua da hari bat zein egoeratan aurki daitekeen jakitea. Egoera horiek grafiko baten bidez adierazi ohi dira,



THREAD ETA RUNNABLE

HARI BAT SORTZEA ETA GAUZATZEA

Javan hariak sortzeko hainbat bide ditugu, Thread klasearen erabilera eta Runnable interfazea konbinatuz, interesatzen zaigunaren arabera:

Thread klasea erabiltzen dugu harengandik heredatzeko. Agian modurik erosoena da, Thread heredatzen duen klase bat automatikoki hari bihurtzen delako. Pega bat du: klase horrek ezingo du beste inoren oinordekorik izan, eta, beraz, gure aplikazioaren arkitekturak hala eskatzen badu, ezingo genuke.

Lehen kasurako aipatu berri dugun muga badugu, Runnable interfazea inplementatu dezakegu, gu inplementatuta gauden klaseak, gainera, inolako mugarik gabe heredatu ahal izateko. **Hari klasearekin zuzenean lan egiteko modua pixka bat aldatzen du.** Bestalde, hari bat ere sor dezakegu klase anonimo bat erabiliz. **Ez da gomendatzen den metodo bat, baina, kasu batzuetan, hari-klaseak egitura zehatzik ez duenean, nahiko erosoa da horrela egitea.**



AZTERTU

1. ¿Qué pasa si ejecutas varias veces el código de los ejemplos? ¿Siempre ocurre lo mismo?
2. ¿Existe alguna manera de asegurar que todo se va a ejecutar en un orden concreto?

HARIEN SINKRONIZAZIOA

Javaren APIak zenbait metodo ematen ditu Thread klasean aplikazio batean hariak sinkronizatzeko:

- join () Metodo honetara jotzen duen hariaren amaiera espero da.
- Thread.sleep (int) Dei hau exekutatzen duen haria lo egoten da parametro gisa zehaztutako denboran (ms-tan)
- isAlive () Haria oraindik aktibo dagoen egiaztatzen du (ez du exekuzioa amaitu)
- yield () Schedulerri iradokitzen dio beste hari bat exekutatzeko (ez da ziurtatzen)

Interesgarria da hari bat nola gelditu jakitea ere. Kasu horretan, Javako APIak ez zuen aholkatu hasiera batean exekuzioa gelditzeko asmatu zen stop () metodoa.

Horrela, gaur egun, gure hariak gelditzeko modu garbiak geuk ezartzera animatzen gaituzte.

FROGAK

- Probak egin join() gabe. Zein da ezberdintasuna?
- Eta hariak Thread klasetik extends eginda, nola funtzionatuko du?
- Hurrengo adibidea aztertu, programatu eta ikusi bere exekuzioa.

ISALIVE() ADIBIDEA

```java
package IsAlive;

public class APP {

    public static void main(String args[]) {
        Ataza atazaNagusia = new Ataza();
        Thread hariNagusia = new Thread(atazaNagusia);

        AtazaAlive atazaAlive = new AtazaAlive(hariNagusia);
        Thread hariaAlive = new Thread(atazaAlive);

        hariNagusia.start();
        hariaAlive.start();

        System.out.println("Noiz bukatuko dute bi hariek?");
    }
}
```

```java
public class AtazaAlive implements Runnable {  2 usages
    private Thread besteHaribat;  2 usages
    public AtazaAlive(Thread besteHaribat) {  1 usage
        this.besteHaribat = besteHaribat;
    }
    @Override
    public void run() {
        while (besteHaribat.isAlive()) {
            System.out.println("Martxan nago beste haria bizirik dagoen bitartean");
            try {
                Thread.sleep( millis: 500);
            } catch (InterruptedException ie) {
                ie.printStackTrace();
            }
        }
        System.out.println("Beste hariak bukatu du eta ni berarekin");
    }
}
```

```java
public class Ataza implements Runnable {  2 usages

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Ni naiz Ataza Nagusia");
            try {
                Thread.sleep( millis: 500);
            } catch (InterruptedException ie) {
                ie.printStackTrace();
            }
        }
    }
}
```

ARIKETAK

1.Realiza una aplicación de consola que cuente hasta un número determinado (mostrando la secuencia por pantalla) utilizando dos hilos, de forma que cada uno de ellos cuente un rango de números

2.Realiza una aplicación de consola que cuente hasta un número determinado (mostrando la secuencia por pantalla) utilizando un número determinado de hilos. La secuencia de números se repartirá a partes iguales entre todos los hilos de forma que a cada uno se le asigne un rango.

HARIEN EGOERA POSIBLEK

Tabla 2.1. Estados de los hilos y su correspondencia en Java

Estado	Valor en Thread.State	Descripción
Nuevo	NEW	El hilo está creado, pero aún no se ha arrancado.
Ejecutable	RUNNABLE	El hilo está arrancado y podría estar en ejecución o pendiente de ejecución.
Bloqueado	BLOCKED	Bloqueado por un monitor.
Esperando	WAITING	El hilo está esperando a que otro hilo realice una acción determinada.
Esperando un tiempo	TIME_WAITING	El hilo está esperando a que otro hilo realice una acción determinada en un período de tiempo concreto.
Finalizado	TERMINATED	El hilo ha terminado su ejecución.

Lortzen dugu **Thread** klaseko *getState()* metodoaren bitartez.

GURE ARIKETAN EGOERA ADIBIDEA

```
ArrayList<Thread.State> hariEgoera = new ArrayList();
hariEgoera.add (haria01.getState());
haria01.start();
while (haria01.getState()!=Thread.State.TERMINATED){
    if(!hariEgoera.contains(haria01.getState())){
        hariEgoera.add(haria01.getState());
    }
}
hariEgoera.add(haria01.getState());
for(Thread.State egoera:hariEgoera){
    System.out.println(egoera);
}
haria01.join();
```